

JS DOM (Document Object Model)

1. What is DOM?

- a. The DOM stands for *Document Object Model*.
- b. The Document Object Model (DOM) is a programming interface (API) for HTML documents.
- c. It represents the HTML document or page in a hierarchical tree of objects (nodes).
- d. It is a programming interface that allows us to create, change, or remove elements from the document.
- e. It is a *tree-like representation of the HTML document*, making it accessible to languages like JavaScript.

2. Why is DOM required?

- a. Provides a **bridge** between HTML/CSS and JavaScript.
- b. Allows **dynamic content** (changing elements after page load).
- c. Without DOM, JavaScript cannot manipulate web pages effectively.

Example:

- Change background color dynamically.
- Add new elements without reloading the page.
- Validate forms in real-time.

3. Types of DOM:

- a. *Core DOM* ⇒ Standard model for all document types (HTML, XML).
- b. *HTML DOM* ⇒ standard model for HTML documents.

- c. **XML DOM** $\Rightarrow$ standard model for XML documents.
- d. **React DOM** $\Rightarrow$ Special implementation in React to interact with the actual browser DOM. Provides **Virtual DOM** for performance optimization.

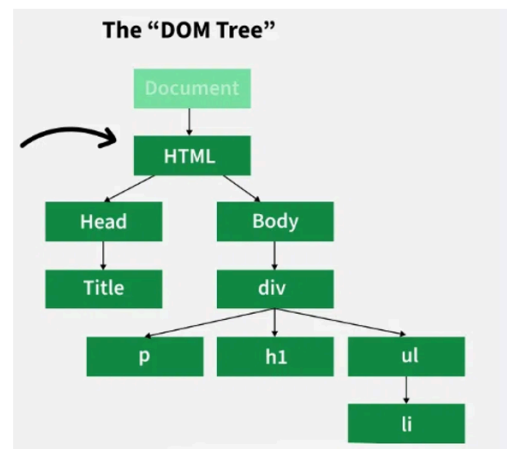
4. Structure of DOM (DOM Tree):

- a. DOM is represented as a **tree structure**.
- b. The top node is a **document**.
- c. Branches represent elements (like `<html>`, `<head>`, `<body>`).
- d. Each HTML tag is a **parent node** with **child nodes**.

```

<html>
  <head>
    <title> Webpage Title</title>
  </head>
  <body>
    <div>
      <h1>This is Heading Tag</h1>
      <p>Some Text Content</p>
      <ul>
        <li> List Item</li>
      </ul>
    </div>
  </body>
</html>

```



5. What is Node and its types?

- a. A **Node** is the basic building block of the DOM tree.
- b. Every piece of HTML (tags, text, comments) is a **node**.
- c. **Types of Nodes:**
 - i. **Element Node** $\rightarrow$ `<p>`, `<div>`, `<img>`
 - ii. **Text Node** $\rightarrow$ Text inside elements.
 - iii. **Comment Node** $\rightarrow$ `<!-- comments -->`
 - iv. **Attribute Node** $\rightarrow$ Attributes like `class="box"`.

- d. **Document Node** → Represents the entire HTML document.
- e. **DocumentFragment Node** → Lightweight, temporary storage of nodes.
- f. **Leaf Node** → Nodes without children (e.g., `<img>`). A node with no child nodes. A text node is often a leaf node.

6. Nodes vs Elements:

- a. **Node**: General term for everything in DOM (elements, attributes, text, comments).
 - b. **Element**: A specific type of node (HTML tag).
- 👉 Every **element** is a **node**, but not every **node** is an element.

7. What is a **document**?

- a. **document** is a **global object** representing the entire DOM.
- b. Provides an **entry point** to access and manipulate the DOM.
- c. Example: `document.getElementById("id")`.

8. Properties of document or Accessing DOM Elements:

- a. Accessing all the elements → `document.all`
- b. Accessing document type → `document.doctype`
- c. Accessing domain name → `document.domain`
- d. Accessing document title → `document.title`
- e. Accessing document body → `document.body`
- f. Accessing document head → `document.head`
- g. Accessing URL of the document → `document.url`

9. Direct Access for DOM Elements:

- a. You can access DOM elements directly.
- b. The `document` object has specific properties for accessing common collections of elements:
 - i. `document.images`: Returns an *HTMLCollection* of all `<img>` elements.
 - ii. `document.links`: Returns an *HTMLCollection* of all `<a>` elements with an *href* attribute.
 - iii. `document.forms`: Returns an *HTMLCollection* of all `<form>` elements.
 - iv. `document.styleSheets`: Returns a *StyleSheetList* of all stylesheets.
 - v. `document.scripts`: Returns an *HTMLCollection* of all `<script>` elements.

10. Indirect Access for Elements:

- a. Indirect access allows us to target the methods that are provided directly by the DOM.
- b. Indirect access methods let you **navigate the DOM tree structure** starting from a known element.
- c. *Some of the types of indirect access methods:*
 - i. `parentNode`:
 - 1. Returns the **parent node** of a selected node.
 - 2. Works on **all nodes** (elements, text, comments).
 - 3. If the node is at the top level (`document`), `.parentNode` will be `null`.

4. For Example →

```
<ul>
    <li id="first">Apple</li>
</ul>
```

```
let li =
document.getElementById("first");
console.log(li.parentNode); //
<ul>...</ul>
console.log(li.parentNode.parentNode);
// <body>...</body>
```

5. Use case: Useful when you need to move upward in the DOM tree.

ii. **childNodes:**

1. Returns a **NodeList** (all types of child nodes including text, comments, and elements).
2. Includes whitespace as **text nodes**.

3. For Example →

```
<ul id="list">
    <li>Apple</li>
    <li>Mango</li>
    <!-- <li>Banana</li> -->
</ul>
```

```
let ul =
document.getElementById("list");
console.log(ul.childNodes); //
NodeList(5) [text, li, text, li, text]
```

4. The extra text nodes come from line breaks / spaces between `<li>`.
5. **Use case:** Use when you want *everything* (including text and comments).

iii. **children:**

1. Returns an **HTMLCollection** of only **element nodes** (ignores text and comments).
2. Safer and cleaner than `.childNodes` for most cases.
3. **For Example** →

```
let ul =
document.getElementById("list");
console.log(ul.children); //
HTMLCollection(2) [li, li]
```
4. **Use case:** Use when you only need HTML elements

11. DOM Access Methods:

- a. These are the primary methods for selecting elements:
 - i. `document.getElementById()`
 - ii. `document.getElementsByClassName()`
 - iii. `document.getElementsByTagName()`
 - iv. `document.getElementsByName()`
 - v. `document.querySelector()`
 - vi. `document.querySelectorAll()`

12. **document.getElementById():**

- a. The `getElementById()` method of the document object returns an HTML element with the specified id.
- b. It will return the HTML element.
- c. Id should be unique.
- d. Returns `null` if no element is found.

e. **Syntax** → `document.getElementById("id");`

f. **For Example** →

```
<p id="demo">Hello World</p>
<script>
  let element = document.getElementById("demo");
  console.log(element); // <p id="demo">Hello
World</p>
</script>
```

g. Styling the Paragraph by using JS *style* property:

```
element.style.border = "2px solid red";
```

13. **document.getElementsByClassName():**

- a. The `getElementsByClassName()` method returns an *HTMLCollection* of elements with the given class name.

b. **Syntax** →

```
document.getElementsByClassName("className");
```

c. **For Example** →

```
<p class="text">Apple</p>
<p class="text">Mango</p>
<script>
  let items =
  document.getElementsByClassName("text");
```

```
console.log(items); // HTMLCollection(2)
[p.text, p.text]
console.log(items[0]); // <p
class="text">Apple</p>
</script>
```

d. HTMLCollection is not an array but it looks like an array.

```
console.log(Array.isArray(items)); // false
```

e. To convert HTMLCollection into an array we have two ways:

i. *By using Array.from() Method:*

```
let htmlCollectionToArray = Array.from(items);
console.log(htmlCollectionToArray);
```

```
console.log(Array.isArray(htmlCollectionToArray)
); // true
```

ii. *By Using Spread Operator:*

```
let htmlCollectionToArray = [...items];
console.log(htmlCollectionToArray);
```

```
console.log(Array.isArray(htmlCollectionToArray)
); // true
```

f. Now you can easily iterate on each and every element inside the HTMLCollection.

14. **document.getElementsByTagName():**

- a. The getElementsByTagName() is a method of the document object to access the HTML elements by their tag name and returns the HTMLCollection.
- b. Tag name can be (div, p, li, etc.)
- c. **Syntax** →

`document.getElementsByTagName ("tagName") ;`

d. **For Example** →

```
<ul>
    <li>One</li>
    <li>Two</li>
    <li>Three</li>
</ul>
<script>
    let list = document.getElementsByTagName("li");
    console.log(list); //HTMLCollection(2) [li, li, li]
    console.log(list[1]); // <li>Two</li>
</script>
```

- e. **Use when:** You want to select elements by their tag.

15. **document.getElementsByName():**

- a. The getElementsByName() returns a **NodeList** of elements with the given name attribute.
- b. Commonly used in forms (radio buttons, inputs).
- c. **Syntax** → `document.getElementsByName ("name") ;`

d. For Example →

```
<input type="radio" name="gender" value="male" /> Male
<input type="radio" name="gender" value="female" />
Female

<script>
  let gender = document.getElementsByName("gender");
  console.log(gender); // NodeList(2) [input, input]
</script>
```

- e. For NodeList you can iterate with forEach Method but does not support the map() method directly because NodeList is not a true array, for that we have to convert the NodeList into an array.

16. **document.querySelector():**

- a. The querySelector() is a method that returns the **first element** that matches a CSS selector.
- b. More flexible than getElementById/Class/Tag.
- c. **Syntax** → `document.querySelector("cssSelector") ;`

d. For Example →

```
<p class="text">Apple</p>
<p class="text">Mango</p>
<script>
  let firstText = document.querySelector(".text");
  console.log(firstText); //<p class="text">Apple</p>
</script>
```

17. **document.querySelectorAll():**

- a. The querySelectorAll() method to select all elements that match a CSS selector or a group of CSS selectors.
- b. Returns a static NodeList.
- c. Supports `forEach()` directly.
- d. **Syntax** →

```
document.querySelectorAll("cssSelector");
```

e. **For Example** →

```
<p class="text">Apple</p>
<p class="text">Mango</p>
<script>
    let allText = document.querySelectorAll(".text");
    console.log(allText); // NodeList(2) [p.text, p.text]

    // Loop with forEach
    allText.forEach((el) => console.log(el.textContent));
</script>
```

18. **What is HTMLCollection?**

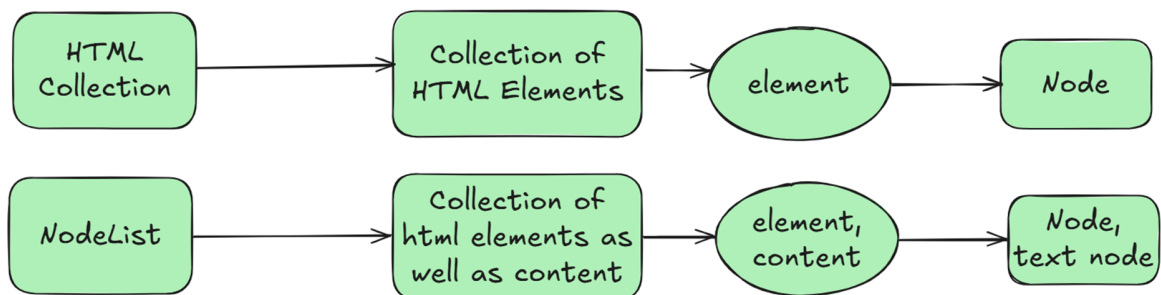
- a. The collection of HTML elements is called HTMLCollection.
- b. It is an array-like object of elements.
- c. Updates automatically if DOM changes.
- d. **HTMLCollection** → `getElementsByClassName()`,
`getElementByTagName()`

19. **What is NodeList?**

- a. The collection of nodes includes text nodes, element nodes, and others called NodeList.
- b. **Array-like object** of nodes.
- c. **Static list** (does not update automatically).

d. **NodeList** → *querySelectorAll()*, *getElementByName()*, and *Node.childNodes*.

20. What is the difference Between HTMLCollection vs NodeList?



21. Styling HTML Elements in JavaScript:

- You can style an element by accessing its *.style* property.
- This property allows you to set inline CSS styles for that element.

c. For Example →

```
document.getElementById("box").style.color = "red";  
document.getElementById("box").style.backgroundColor = "yellow";
```

22. Adding a Class Dynamically:

- You can add a CSS class to an element at runtime using:

- .className* (older way)
- .classList* (modern, preferred)

b. For Example →

```
<p id="para">Hello World</p>  
<script>  
  let p = document.getElementById("para");  
  p.className = "highlight"; //overwrites any existing class  
</script>
```

23. Using `classList` (Recommended):

- `element.classList` is a `DOMTokenList` object that represents all the classes of an element.
- It provides **methods** to safely add, remove, toggle, or check classes without overwriting existing ones.
- For Example →

```
<p id="para" class="text">Hello</p>
<script>
  let p = document.getElementById("para");
  p.classList.add("highlight"); // adds highlight class
  console.log(p.className); // "text highlight"
</script>
```

24. Common `classList` Methods:

Method	Description	Example
<code>add("class")</code>	Adds a class	<code>p.classList.add("active")</code>
<code>remove("class")</code>	Removes a class	<code>p.classList.remove("active")</code>
<code>toggle("class")</code>	Adds if missing, removes if present	<code>p.classList.toggle("dark-mode")</code>
<code>contains("class")</code>	Checks if class exists → returns true/false	<code>p.classList.contains("active")</code>
<code>replace("old", "new")</code>	Replaces one class with another	<code>p.classList.replace("old", "new")</code>
<code>item(index)</code>	Returns class name at given index	<code>p.classList.item(0)</code>

25. Practical Example of classList:

index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Active Link Example</title>
    <style>
      body {
        font-family: Arial, sans-serif;
      }
      nav ul {
        list-style: none;
        display: flex;
        gap: 20px;
      }
      a {
        text-decoration: none;
        color: black;
        font-weight: bold;
        transition: color 0.3s ease;
      }
      a.active {
        color: red;
        border-bottom: 2px solid red;
      }
    </style>
  </head>
  <body>
    <nav>
      <ul>
        <li><a href="#" class="nav-link">Home</a></li>
        <li><a href="#" class="nav-link">About</a></li>
        <li><a href="#" class="nav-link">Services</a></li>
        <li><a href="#" class="nav-link">Contact</a></li>
      </ul>
    </nav>

    <script src="script.js"></script>
  </body>
</html>
```

script.js

```
#!/ Select all nav links
let links = document.querySelectorAll(".nav-link");

links.forEach((link) => {
  link.addEventListener("click", function (event) {
    event.preventDefault(); //! Prevent page refresh

    //! Remove 'active' from all links
    links.forEach((l) => l.classList.remove("active"));

    //! Add 'active' to the clicked link
    this.classList.add("active");
  });
});
```

Output:

Before clicking on the nav link.

Home About Services Contact

After clicking on the navlink.

Class -> active will be added -> css will apply for .active class.

Home About Services Contact